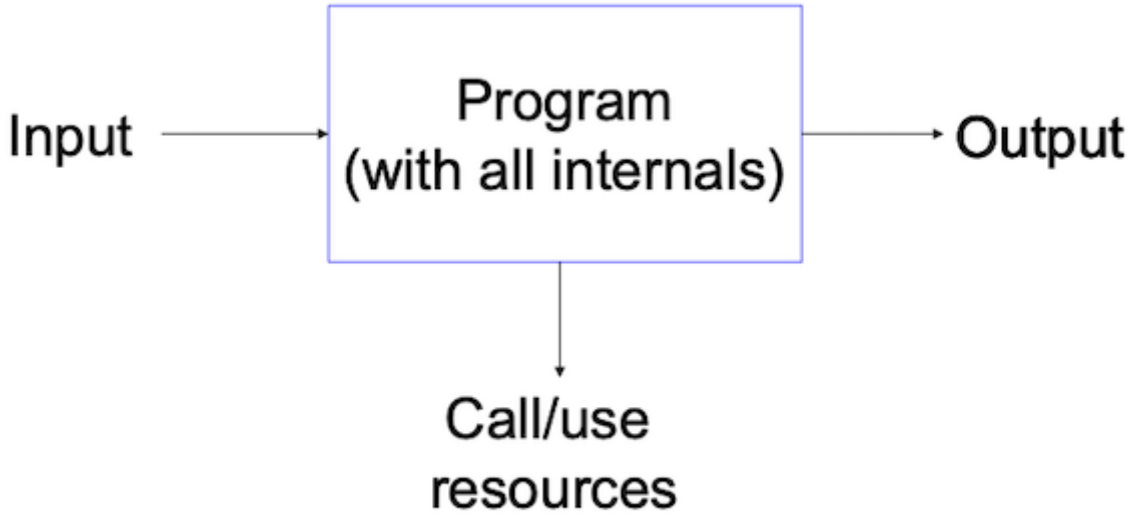
Output Information Judiciously

The Big Picture

- not to produce **outputs** that make other programs downstream vulnerables.
- **input validation**
- **resources**: what assumptions to make



Schema after David A. Wheeler.

Minimize Feedback

- Unnecessary information can help attackers.
- **Example**: Login failure messages.
  - **Safe:** "Login failed."
  - **Unsafe:** Incorrect username vs. Incorrect password (reveals valid usernames).
    - Distinguishing between the two: ability to deduce correct usernames on system.

Avoid Fingerprinting

- **Do not expose system or program versions.**
- Attackers use version info to exploit known vulnerabilities.
  - aovid to identify old/unpatched program

Log, Don't Print, Errors

- Informative messages are useful For administrator, not untrusted user
- Detailed information about error cause should be written to logs.
  - Log permissions can be restrictive
- Design audit system jointly with program
  - Allows systematic error logging
  - Avoid accidental **debug** output

Prevent Denial of Service (DoS) on Output

- **Attackers may control output channels** (e.g., files, network connections).
- Program waiting for output may get stuck
  - => **denial-of-service attack!**
- **Mitigations:**
  - Set timeouts on network outputs.
  - Release locks before writing output (prevents blocking).

Filter Output Defensively

- **Input filtering is critical, but output filtering also adds security.**
- **Example: HTML encoding to prevent XSS attacks.**

```
&lt;script&gt;alert("XSS")&lt;/script&gt;
```

Information Flow Foundations

Information Leaks

- Information leak is a disclosure on information
- Attack on Confidentiality.
- **Causes:**
  - Poor design.
  - Implementation flaws.
  - Misunderstood importance
  - Lack of privacy awareness.

Example of information leak

```c
#include <stdio.h>

int main() {
    int secret = 42;        // Confidential data
    int public = 0;         // Publicly observable variable

    if (secret == 42) {
        public = 1;
    } else {
        public = 0;
    }

    printf("Public value: %d\n", public);  // Leaks whether secret was 42
    return 0;
}
```

Information flow

- **Definition of information flow:** transfer of information between two variables in some process (execution context)
- **Undesirable**: leaking secret information into publicly observable variable
- Simplest scenario:
  - two security levels, high (**H**) and low (**L**)
  - observing information/variable at low
  - level **L** should leak no information about level **H**

Multi-Level Security (MLS)

- Inspired by **military** classification.
- Bell-LaPadula Model
  - No Read Up (NRU): Lower-level users cannot read higher-level data.
  - No Write Down (NWD): Higher-level users cannot write to lower levels.

Non-Interference (part I)

- Property formalizing information flow (Goguen & Meseguer, 1982)
- Low inputs: public information that an **external observer** (e.g., a regular user or an attacker) is allowed to see.
- High inputs: confidential or secret information (e.g., **passwords**, **private messages**).
- The two processes do not interfere:
  - sequence of low inputs will produce same sequence of low outputs, regardless of any high-level inputs.
  - Secret (high) data must not affect what an attacker can see (low outputs).

Non-Interference (part II)

- information flow control is not about stopping the attacker from guessing secrets
- it's about ensuring that the program's observable behaviour (outputs, timing, crashes, etc.) never changes based on secret data.
- No matter what the attacker does with low (public) inputs — even guessing a secret — the low outputs they observe will be the same, regardless of the actual secret.

What happens if the attacker guesses the password?

- The system violates noninterference:
  - the attacker might observe something different when guessing correctly
  - A different output message ("Login successful")
  - A different timing pattern (it takes longer to reject a bad password)
  - A memory leak (e.g., Heartbleed)
- The system enforces noninterference:
  - even a correct guess should not reveal any observable difference
  - unless the guess is made through a secure channel that is meant to depend on high inputs

Example of non-interference violation in C

```c
#include <stdio.h>

void process(int high, int low) {
    int output = low;

    if (high > 0) {
        output += 1;  // secret influences public output!
    }

    printf("Output: %d\n", output);  // this is a low (public) output
}
```

Example of non-interference violation in C

```c
if (strcmp(input_password, secret_password) == 0) {
    printf("Welcome!\n");  // low output changes if secret is guessed
} else {
    printf("Wrong password.\n");
}
```

Example of non-violation in C

```c
int result = strcmp(input_password, secret_password);
// process result internally, don't expose it to public output
printf("Authentication result is logged.\n");
```

Side Channels

Side vs. Covert Channels

- Side channel: unintended information transfer due to implementation flaws (rather than design flaws)
  - Unintended, external
- Covert channel: not intended for transfer of information, but allows information flow not intended by security policy.
  - Intended, internal

Covert vs. Side Channels

- Side channel
- passive information transfer by observing some system activity
  - timing channels
  - storage channels
  - power analysis
- Covert channel

- usually involves abusing the system
  - communication between an injected Trojan and the attacker
  - malware communicating with a backdoor

Timing Attacks

- Attacker learns something about secret system state by measuring time taken by some operation.
- **Example:** SSH Timing Attack (Song, Wagner & Tian, 2001).
  - SSH transmitted **one password character per packet**.
  - Timings leaked keyboard positioning.
- **Example:** String comparison timing.

```
if (typed_passwd[i] != actual_passwd[i]) return fail;
```

  - Attackers **learn passwords one character at a time** by measuring response time.

Timing String Comparisons

- Password leak: TENEX operating system
- Timing differs based on count of correct characters at the start
- Find password characters one by one!

```
for i from 0 to len(typed_passwd):
  if i >= len(actual_passwd) then return fail
  if typed_passwd[i] != actual_passwd[i] then return fail
  if i < len(actual_passwd) then return fail
return success!
```

The TENEX operating system was an influential time-sharing operating system developed in the early 1970s.

Storage Channel Attacks

- Information obtained from (other) data
- **Existence** of some data (a file, even if contents are unreadable or encrypted)
- **Length** of transmitted message (may be different for success or failure)
- **Metadata** (find out source or time when data produced)
- Actual data (ICMP error packets may include information about target system)

Storage Channel Attacks

- **Metadata leaks:** File existence, message length, timestamps.
- **Example:** ICMP error packets exposing internal system structure.

Power Analysis

- Power consumed by a computing device may depend on:
  - Quantity of data processed
  - Data values (different for 0 and 1 bits)
  - Information flow can be analyzed and quantified at programming language level

Power Analysis

- **Power consumption analysis** can reveal cryptographic keys.
- **Some bits use more power than others**, leaking data.

Meltdown and Spectre

Major flaws in 2028

- Hardware-level vulnerabilities affecting most major processors
- Both break basic memory isolation properties needed for security
- Rely on:
  - Out-of-order execution (Meltdown)
  - Speculative execution (Spectre)
- Underlying techniques evolved over time, attacks/fixes made public January 2018.

```
Out-of-order execution is a hardware optimisation technique used in modern
CPUs to improve performance by executing instructions as soon as their
operands are ready, rather than strictly following the original program
order.
```

Meltdown

- **Exploits out-of-order execution.**
- **Allows user-space access to kernel memory.**
- **Prevention:** Kernel Page Table Isolation (KPTI).
- Load of data from privileged address into a temporary register can happen, but data is discarded, not visible at instruction level
- But execution effects are observable - in particular, effects on cache

> The Meltdown vulnerability, disclosed in January 2018, was a critical CPU security flaw that allowed unprivileged user-space programs to read kernel memory – which should have been completely inaccessible.

Meltdown and Spectre in 3 Minutes

Meltdown and Spectre

Meltdown Sample Code

```
retry:
  mov al, byte [rcx]; // Read 1 byte from a kernel address (causes fault)
  shl rax, 0xc ;      // rax = rax * 4096
  jz retry ;          // next: out of order
  mov rbx, qword [rbx + rax]
```

- Attempts to read a secret byte from kernel memory
- Illegal for user space → should trigger a segmentation fault.
- However, on **speculative execution**, this may temporarily succeed before the fault is processed.
- `rax = [ 56-bit high ][ 8-bit low = al ]`
- `4096 * al` will lead to different pages
- If cache line later found in memory, means page was loaded (side channel)
- Can find value at arbitrary address (rcx)

Attacker

```
for (int i = 0; i < 256; i++) {
    if (timing(probe[i * 4096]) is fast)
        secret = i;
}
```

Meltdown prevention

- Don’t map kernel memory to user space (except a few critically needed addresses) on Linux: Kernel Page Table Isolation
- Randomize kernel addresses (KASLR) makes attack hard for addresses that have to be mapped

Spectre

- **Exploits speculative execution.**
- **Mistrained branch prediction leads to unauthorized memory access.**
- **Prevention:** Modify CPU instruction behavior (performance cost).

Files: Unix Specifics

File Permissions

- Read-write-execute for user, group, others `-rwxr-xr-x root root /usr/bin/gcc`
- File creation permissions depend on `umask` of process
- Permission bit set if allowed both by process umaskand file-creation mode
- umaskinherited from parent process but can be explicitly set

Directory Permissions

Two distinct notions—and permission bits

- `x`: enter directory (chdir() system call) and access file inode information (stat() call), needed for subsequent file operations also called search permission

- `r`: read directory (open(), readdir() calls), needed to enumerate files in directory (when their names are not a priori known)

Security - Directory Permissions

- `w` bit: allows writing the directory Can delete or rename files, regardless of permission on actual file! (Security: integrity vs. availability)
- `r` bit: allows finding out filenames Important (e.g., for web server) Crawler could browse directories having permissions of server and discover files!

Dangers of Privilege Misuse

- Privileged programs can be tricked into a **confused deputy** situation
- Misuse to access files that are normally beyond user's reach
- Programmer calls library function without knowing all behaviors due to configuration

Sample FreeBSD Flaw

```
if (newcommand == NULL && !quiet_login && !options.use_login) {
  fname = login_getcapstr(lc, "copyright", NULL, NULL);
  if (fname != NULL && (f = fopen(fname, "r")) != NULL) {
    while (fgets(buf, sizeof(buf), f))
      fputs(buf, stdout);
    fclose(f);
  }
}
```

Vulnerability Discussion

- Code from privileged openssh calling non- privileged library libutil
- Gets login capabilities from configuration file ~/.login.conf, which is user controlled
- If name of copyright file maliciously set to `/etc/master.passwd`, contents is printed, although file is only readable by root
- Combination of **confused deputy** and violating least common mechanism